SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

AIM :
Implement the A* Search algorithm for solving a pathfinding problem.
Implement the Recursive Best-First Search algorithm for the same problem.
Compare the performance and effectiveness of both algorithms.

CODE:

```
import heapq
import matplotlib.pyplot as plt
import networkx as nx

graph = {

    'Tata Power Residential Colony': {
        'Raghuleela Mega Mall': 2.4,
        'Thakur College': 2.8
    },

    'Raghuleela Mega Mall': {
        'Charkop': 2.2,
        'Malad-Marve Road': 3.1
    },

    'Thakur College': {
        'Malad-Marve Road': 3.4
    },

    'Charkop': {
        'Vishals Magic Cricket Academy': 2.3
    },

    'Malad-Marve Road': {
        'Vishals Magic Cricket Academy': 2.0
    },

    'Vishals Magic Cricket Academy': {
        'Children Park and Gymnasium': 0.03
    },

    'Children Park and Gymnasium': {}
}
```

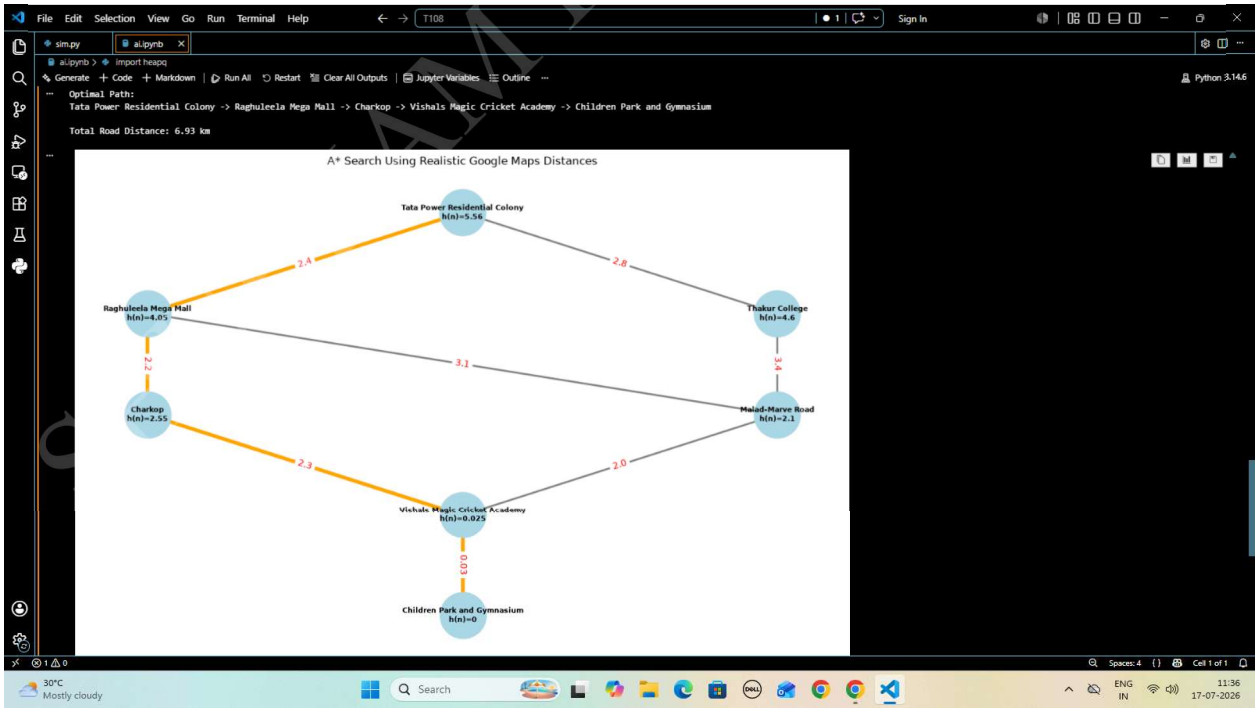
```
heuristics = {

    'Tata Power Residential Colony': 5.56,
    'Raghuleela Mega Mall': 4.05,
    'Thakur College': 4.60,
    'Charkop': 2.55,
    'Malad-Marve Road': 2.10,
    'Vishals Magic Cricket Academy': 0.025,
    'Children Park and Gymnasium': 0
}


node_positions = {

    'Tata Power Residential Colony': (4,8),
    'Raghuleela Mega Mall': (2,6),
    'Thakur College': (6,6),
    'Charkop': (2,4),
    'Malad-Marve Road': (6,4),
    'Vishals Magic Cricket Academy': (4,2),
    'Children Park and Gymnasium': (4,0)

}


def a_star_search(graph, heuristics, start, goal):

    priority_queue = []
    heapq.heappush(priority_queue, (heuristics[start], start, [start], 0))

    visited = set()

    while priority_queue:

        f_score, current, path, g_score = heapq.heappop(priority_queue)

        if current in visited:
            continue
```

```
        visited.add(current)

        if current == goal:
            return path, g_score

        for neighbor, edge_weight in graph[current].items():

            if neighbor not in visited:

                new_g = g_score + edge_weight
                new_f = new_g + heuristics[neighbor]

                heapq.heappush(
                    priority_queue,
                    (new_f, neighbor, path + [neighbor], new_g)
                )

    return None, float("inf")


start = "Tata Power Residential Colony"
goal = "Children Park and Gymnasium"

optimal_path, total_distance = a_star_search(
    graph,
    heuristics,
    start,
    goal
)

print("Optimal Path:")
print(" -> ".join(optimal_path))

print("\nTotal Road Distance:", round(total_distance,2), "km")


G = nx.DiGraph()

for node, neighbors in graph.items():
```

```
    for neighbor, weight in neighbors.items():
        G.add_edge(node, neighbor, weight=weight)

plt.figure(figsize=(12,8))

path_edges = list(zip(optimal_path, optimal_path[1:]))
normal_edges = [edge for edge in G.edges() if edge not in path_edges]

nx.draw_networkx_nodes(
    G,
    node_positions,
    node_size=2600,
    node_color="lightblue"
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=normal_edges,
    width=2,
    edge_color="gray",
    arrows=True
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=path_edges,
    width=4,
    edge_color="orange",
    arrows=True
)

labels = {
    node: f"{node}\nh(n)={heuristics[node]}"
    for node in G.nodes()
}

nx.draw_networkx_labels(
    G,
```

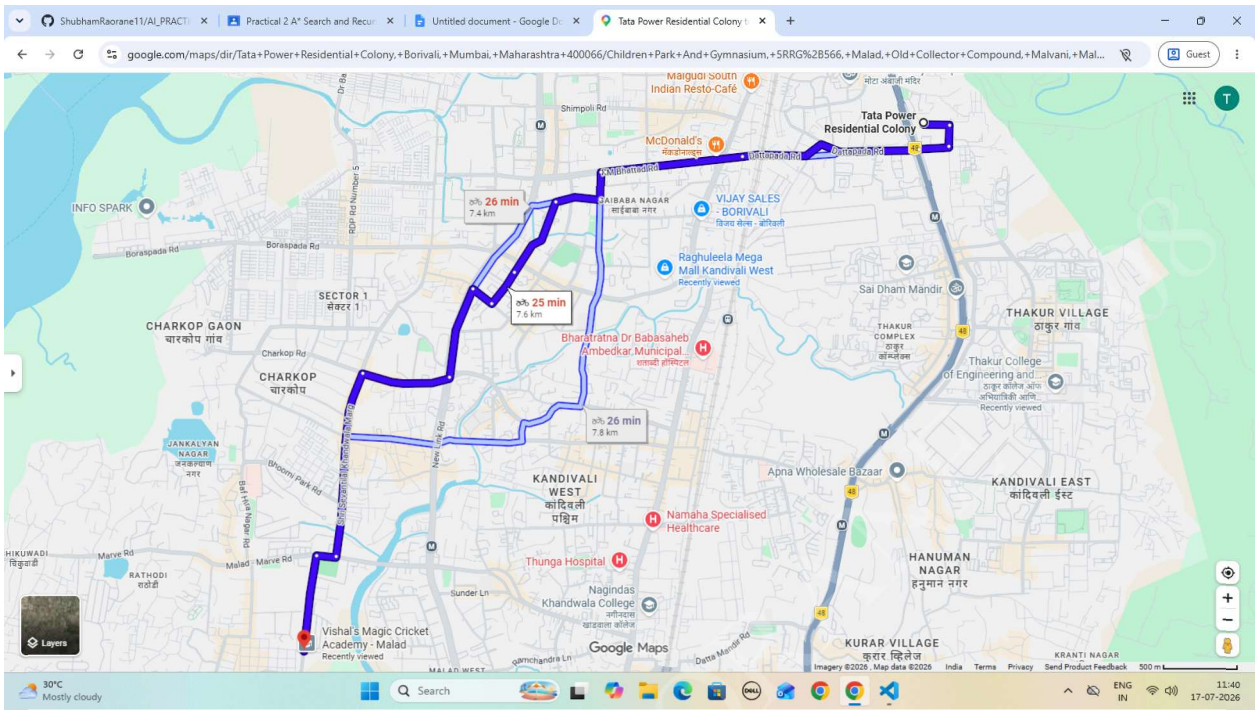
```
    node_positions,
    labels=labels,
    font_size=8,
    font_weight="bold"
)

edge_labels = nx.get_edge_attributes(G, "weight")

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=edge_labels,
    font_color="red"
)

plt.title("A* Search Using Realistic Google Maps Distances")

plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

MAP:



SHUBHAM RAORANE T108